

Practical 10 - Part 1

Supervised Learning

In this laboratory tutorial, you will work with Supervised Machine Learning. In particular, you will learn how to implement Logistic Regression and Decision Tree models for classification problems. You will first use some historical data, split it into training and testing datasets before running the classifier, and finally evaluate the classifier using confusion matrix.

You will need Jupyter notebook for Python 3.

Useful resources:

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>

Exercise 1: Classification

Suppose you are asked to build an email spam classifier.

Task 1. What could be the training data, features and the output label? How would you evaluate the classifier?

Exercise 2: Confusion Matrix

Each of the following confusion matrices relate to a classification model.

N=43	Predicted Class (0)	Predicted Class (1)
Actual Class (0)	3	20
Actual Class (1)	2	18

Confusion Matrix 1

N=56	Predicted Class (0)	Predicted Class (1)
Actual Class (0)	28	2
Actual Class (1)	1	25

Confusion Matrix 2

N=47	Predicted Class (0)	Predicted Class (1)
Actual Class (0)	3	18
Actual Class (1)	24	2

Confusion Matrix 3

Task 1. For each confusion matrix, calculate the accuracy and error to two decimal places. Interpret your results.

Task 2. For each confusion matrix, calculate sensitivity, specificity, precision, recall, and F measure. Interpret your results.

Exercise 3: Running Logistic Regression model in Python

Using scikit-learn library we will build a logistic regression model in python. We will first generate and split a dataset into training dataset and test dataset. We will then train the logistic regression model (as a classifier), make some predictions, and finally perform the evaluation.

Step 1. Open Jupyter notebook

Step 2. Import the following library:

```
from sklearn.datasets import make_classification
```

It helps to generate a dataset that can be used for logistic regression

Step 3. Next, we will generate a dataset

```
X,Y=make_classification(n_samples=20,n_features=1,n_classes=2,n_clusters_per_class=1,flip_y=0.03,n_informative=1,n_redundant=0,n_repeated=0)
```

X is the feature, and Y is the categorical output, with class: 0 and 1

Basically, we have 20 random values and each value is categorised as 0 or 1

Step 4. Let us see what we have for X and Y

```
print(X)
```

```
print(Y)
```

These are some randomly generated data grouped into two classes.

Step 5. We can also visualise our data. Import matplotlib to visualise the data.

```
from matplotlib import pyplot as plt
```

```
plt.scatter(X,Y,cmap = 'Blue')
```

```
plt.show()
```

Step 6. Split the data into training dataset (80%), and test dataset (20%).

In terms of the 20% test dataset we will test whether the logistic regression model that we are going to build is predicting well or not. We need to import train_test_split

```
from sklearn.model_selection import train_test_split
```

```
X_train,X_test,Y_train,Y_test=train_test_split(X,Y,test_size=0.20)
```

Step 7. See what is there in X_train and Y_train.

```
X_train
```

There should be 16 values (relates to feature). This is 80% of the data that is trained.

```
Y_train
```

There should be 16 values (relates to output: as 0 or 1), this is 80% of the data that is trained.

We can observe that in the X_train when a value for x is negative, if you map it with the value of Y_train, it is 0. When a value for x is positive or 0, then its Y_train value is 1.

Step 8. See what is there in X_test (as feature) and Y_test (as actual output)

X_test

There should be 4 values (20% of the testing data)

Y_test

There should be 4 values (20% of the testing data)

Similar to Step 7, map X_test with Y_test, and observe.

We will now check whether the logistic regression model that we are going to build correctly predicts the output (as 0 or 1) or not. In other words, is it predicting the output same as the actual output (as in Step 8). To do this we need both the training and testing data.

Step 9. Let's perform Logistic Regression.

```
from sklearn.linear_model import LogisticRegression
```

```
logistic_regression_model=LogisticRegression()
```

Here, we created a Logistic Regression object. Now, train the model on 80% of the data. For this, we need to fit the object to our data.

```
logistic_regression_model.fit(X_train,Y_train)
```

Step 10. We need to test the trained model using the test dataset.

```
prediction=logistic_regression_model.predict(X_test)
```

Print the predicted output.

```
print(prediction)
```

Step 11. Compare the predictions with Y_test (actual output)

Y_test

Check if there is any difference between the actual and predicted outputs. If not, this means the logistic regression model is predicting well. In reality, the regression model will still make some errors (which we will find out in Step 13))

Step 12. Similar to Linear Regression, we can calculate the intercept and the regression coefficients.

```
intercept=logistic_regression_model.intercept_
```

```
print(intercept)
```

```
coeff=logistic_regression_model.coef_
```

```
print(coeff)
```

Step 13. Evaluate the Logistic Regression model using a confusion matrix

```
from sklearn.metrics import confusion_matrix
```

```
confusion_matrix(Y_test,prediction)
```

What is your output?

Calculate Accuracy manually using the generated confusion matrix.

Calculate Accuracy using scikit learn.

```
from sklearn.metrics import accuracy_score
```

```
model_accuracy=accuracy_score(Y_test,prediction)
```

```
model_accuracy
```

Does it match with the value calculated manually?

Step 14. In Step 13, the accuracy value is found to be close or equal to 1. This means the Logistic Regression model is predicting well and may be considered positively.

Step 15. Since the Logistic Regression model is predicting well, we can now predict a new instance of data. Let's find the value of y with x = -2.069

```
logistic_regression_model.predict([[-2.069]])
```

What do you get as output?

Exercise 4: Running Decision Tree model in Python

Using scikit-learn library we will build a decision tree classifier in python. We will first read a dataset given as a csv file and learn the decision tree from this data, and then make some predictions.

In the lecture, we discussed the example of playing tennis while discussing the decision tree learning. This data contains records of 14 days with four input attributes (outlook, temperature, humidity, and wind) related to the weather condition on a day, and one output class (playtennis), indicating whether it is a good day to play tennis or not. We are going to learn our decision tree from this dataset and going to apply the learned classifier to predict whether we should play tennis on a day with given weather conditions or not. The dataset is given in the form of a csv file called **playtennis.csv**.

Step 1. Open Jupyter notebook

Step 2. Import the following libraries:

```
import numpy as np
import pandas as pd
```

Step 3. Next, read the dataset playtennis.csv and show its contents.

```
PlayTennis = pd.read_csv("playtennis.csv")
PlayTennis
```

Step 4. All our attribute values are categorical in nature. It is generally easy to implement a machine learning algorithm with numeric values. Therefore, we would encode all the given values to some numbers using LabelEncoder.

```
from sklearn.preprocessing import LabelEncoder
Le = LabelEncoder()
PlayTennis['outlook'] = Le.fit_transform(PlayTennis['outlook'])
PlayTennis['temperature'] = Le.fit_transform(PlayTennis['temperature'])
PlayTennis['humidity'] = Le.fit_transform(PlayTennis['humidity'])
PlayTennis['wind'] = Le.fit_transform(PlayTennis['wind'])
PlayTennis['playtennis'] = Le.fit_transform(PlayTennis['playtennis'])
```

Step 5. See the contents of the data now.

```
PlayTennis
```

Step 6. Let's split the input/independent attribute values that will be used for training and their corresponding output/prediction attribute values. X holds the input attributes used for training, and y holds all the decisions.

```
y = PlayTennis['playtennis']
X = PlayTennis.drop(['playtennis'],axis=1)
X = X.drop(['day'],axis=1)
```

Step 7. From scikit-learn we will use tree, to create our decision tree classifier and fit the model.

```
from sklearn import tree  
  
clf = tree.DecisionTreeClassifier(criterion = 'entropy')  
  
clf = clf.fit(X, y)
```

Step 8. We can visualize the tree using tree.plot_tree.

```
plt.figure(figsize=(8,8))  
  
tree.plot_tree(clf,  
               feature_names=["outlook", "temperature", "humidity", "wind"],  
               class_names=["No", "Yes"],  
               filled=True,  
               fontsize=10)  
  
plt.show()
```

Step 9. Now let's use our trained model to predict our train data X, and compare the predicted output with their actual values.

```
X_pred = clf.predict(X)  
  
X_pred == y
```

Summary

In this tutorial you have learned how to use Logistic Regression and Decision Tree models for classification problems, and also how to evaluate a classifier.

Reading

Textbook: Elements of Statistical Learning

- <https://web.stanford.edu/~hastie/Papers/ESLII.pdf>
- Chapter: 13